COMP20280
Treap Data Structure

Dermot Yang Li Weihao Robbie Devilly Thehan Arunadewa

April 2026

1 Introduction

In this project, we implement and evaluate a Treap, which is a type of binary tree data structure. A Treap combines two ideas: it keeps elements ordered like a binary search tree (BST), and it also assigns each element a random priority, like in a heap. This combination helps keep the tree balanced on average, so operations such as searching, inserting, and deleting can be performed efficiently in about $O(\log n)$ time. Unlike other balanced trees such as AVL or red-black trees, a Treap does not require complicated rules to maintain balance.
Our implementation, `TreapMap<K,V>`, is based on the `TreeMap` class provided in the course. We compare its performance with our own `AVLTreeMap` and Java's built-in `TreeMap` using different input sizes (from 100 to 10,000 elements) and different types of data (random, ascending, descending, and partially sorted). We also compare `TreapSort` with four other sorting algorithms.
The results show that the Treap performs efficiently, with an average time of $O(\log n)$ for operations across all input types. It performs especially well on sorted data, where AVL trees tend to require more rotations to stay balanced.

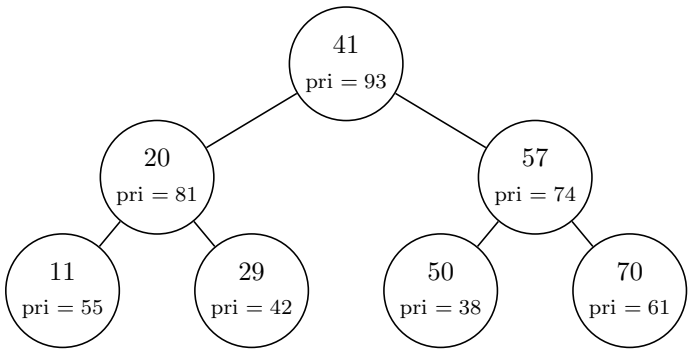
2 How a Treap Works

2.1 Definition

Let X be a set of n items. Each item has a *key* (which can be ordered, like numbers) and a randomly assigned *priority*. A Treap is a binary tree built from these items that follows two rules:

- **BST property (by key):** For any node x, all keys in the left subtree are smaller than $x.key$, and all keys in the right subtree are larger.
- **Max-heap property (by priority):** For any node x, its priority is less than or equal to the priority of its parent.

These two properties work together to keep the tree organised and balanced. The BST property allows efficient searching, while the heap property (based on random priorities) helps prevent the tree from becoming too skewed.
If all keys and priorities are different, the structure of the Treap is fixed. The node with the highest priority becomes the root, and the rest of the nodes are placed according to the BST rule based on their keys.
Because priorities are assigned randomly, the tree is balanced on average, which means its height is typically $O(\log n)$. This is what allows Treaps to perform operations like search, insertion, and deletion efficiently.



In-order traversal: 11 → 20 → 29 → 41 → 50 → 57 → 70 (keys always emerge in sorted order)

Figure 1: A Treap storing seven items. Each node shows its **key** (large) and its randomly assigned **priority** (small). The tree follows two rules: the keys are arranged like in a binary search tree (smaller values on the left, larger on the right), and the priorities follow a max-heap structure (each node has a higher priority than its children). Because the priorities are random, the tree stays balanced on average, giving it a height of about $O(\log n)$.

2.2 Insertion

To insert a new item z into a Treap T:

1. Assign z a random priority.
2. Insert z into the tree using the normal BST rule (based on its key). At this point, the BST property is satisfied, but the heap property might be broken.
3. If z's priority is higher than its parent's, move it upward using rotations (left or right, depending on its position) until the heap property is restored or z becomes the root.

Each rotation keeps the BST structure correct while fixing the priority order. On average, fewer than 2 rotations are needed per insertion.

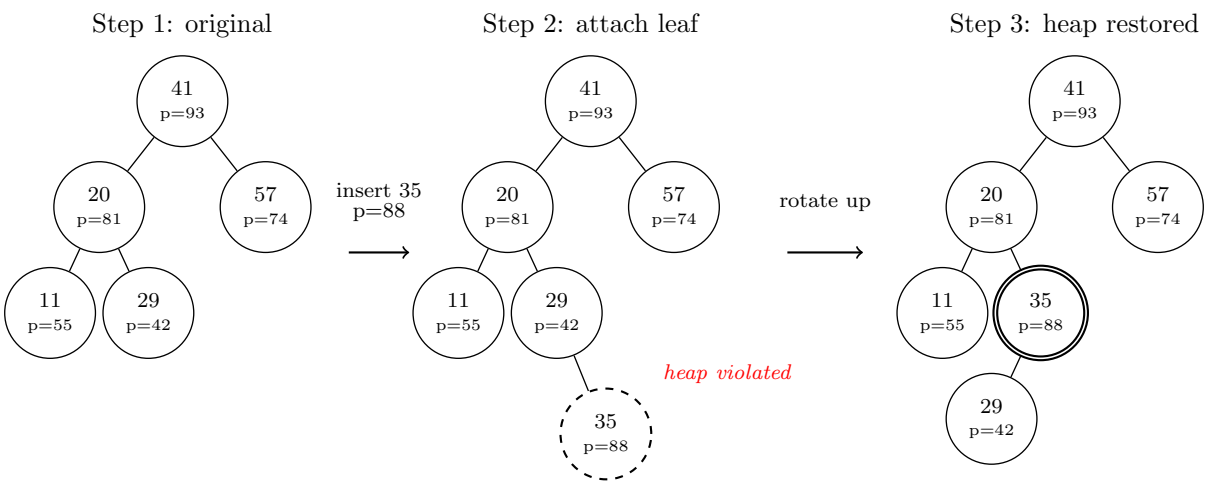


Figure 2: Inserting key 35 with priority 88 into a Treap. In Step 2, node 35 is attached as a leaf in the correct BST position (right child of 29), but its priority (88) is higher than its parent's (42), violating the heap property. In Step 3, node 35 is rotated upward until the heap property is restored. The double-bordered circle marks the newly inserted node after rotation.

2.3 Deletion

To delete a node x from a Treap:
First, find x in the tree using the standard BST search (based on its key). Then, move x downward by repeatedly rotating it with one of its children. At each step, swap it with the child that has the higher priority so that the heap property is maintained. Once x has been moved down to a leaf (a node with no children), it can simply be removed from the tree.

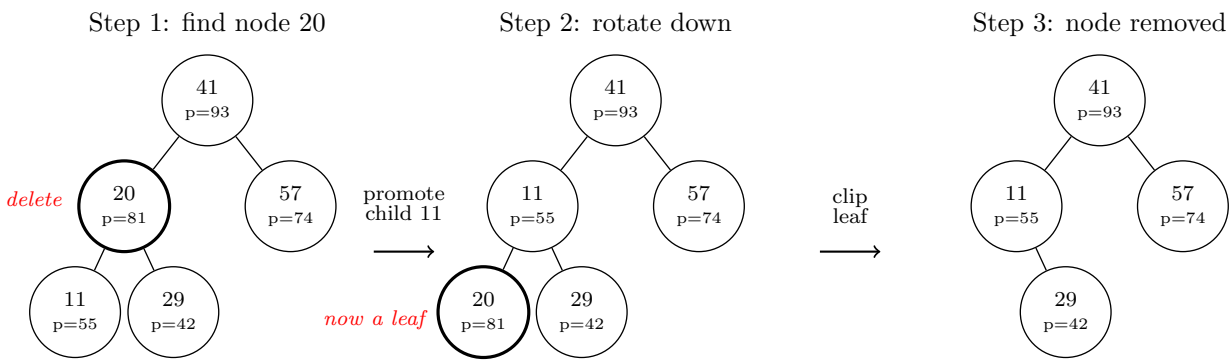


Figure 3: Deleting node 20 from a Treap. In Step 2, node 20 is moved downward by rotating it with its child that has the higher priority (11, p=55), keeping the heap property correct. Once node 20 becomes a leaf, it is removed from the tree, leaving a valid Treap in Step 3.

3 Implementation Design Decisions

3.1 Extending `TreeMap`

`TreapMap<K,V>` is built by extending the course-provided `TreeMap<K,V>` class, which already includes the basic binary search tree structure, such as storing nodes, searching by key, and methods like `subMap`, `ceilingEntry`, and `floorEntry`. We only override the methods that need different behaviour: `rebalanceInsert`, `rebalanceDelete` (which is left empty since deletion is handled separately), and `remove`.
This design keeps the public interface the same as `TreeMap`, so no changes are needed in client code when switching implementations. It also avoids rewriting code that is already provided, such as tree traversal and iterators.

3.2 Insertion Rebalancing

`rebalanceInsert` is called by the parent class immediately after the new leaf is attached. It assigns a random priority and rotates upward: The rotate(p) call in the parent class selects left or right rotation automatically based on whether p is a left or right child, so no explicit case analysis is needed.

```
@Override
protected void rebalanceInsert(Position<Entry<K, V>> p) {
    setPriority(p, random.nextInt());
    while(!isRoot(p)) {
        Position<Entry<K, V>> par = parent(p);
        if(priority(p) > priority(par)) {
            rotate(p);
        } else {
            break;
        }
    }
}
```

Listing 1: Insertion rebalancing in `TreapMap`.

3.3 Deletion

In a standard Binary Search Tree, deleting a node with two children usually involves replacing it with its in-order successor. In a Treap, deletion is easier. The node is first rotated downward until it becomes a leaf. At each step, the child with the higher priority is rotated up. This keeps the heap property valid throughout the process. Once the node becomes a leaf, it can be removed directly.

```java
@Override
public V remove(K key) throws IllegalArgumentException {
    checkKey(key);
    Position<Entry<K, V>> p = treeSearch(root(), key);

    if(isExternal(p)) {
        rebalanceAccess(p);
        return null;
    }
    V old = p.getElement().getValue();
    while(isInternal(left(p)) || isInternal(right(p))) {
        Position<Entry<K, V>> leftC = left(p);
        Position<Entry<K, V>> rightC = right(p);

        boolean hasLeft = isInternal(leftC);
        boolean hasRight = isInternal(rightC);

        Position<Entry<K,V>> highChild;
        if(hasLeft && hasRight) {
            highChild = (priority(leftC) > priority(rightC)) ? leftC :
                rightC;
        } else {
            highChild = hasLeft ? leftC : rightC;
        }

        rotate(highChild);
    }
    Position<Entry<K, V>> leaf = isExternal(left(p)) ? left(p) : right(p);
    remove(leaf);
    remove(p);
    return old;
}
```

Listing 2: Deletion in `TreapMap`.

rebalanceDelete is left empty because all rebalancing during deletion is handled by the rotation loop above.

3.4 Correctness Testing

Two helpers were added to support unit testing.

- `isValidTreap()` recursively verifies that the tree satisfies both the BST ordering on keys and the heap ordering on priorities in $O(n)$ time. It is invoked after every stress test, where a stress test consists of 1000 randomly interleaved insertions and deletions.
- `height()` returns the actual height of the tree and is used by `testSortedInsertionStaysBalanced`. This test inserts 1000 keys in ascending order and asserts that the resulting height remains below 80. Since a sorted insertion sequence would produce a linear chain in an ordinary BST, passing this test confirms that the randomised priorities successfully prevent degenerate growth.

4 Performance Comparison

4.1 Methodology

All benchmarks were run with 5 timed iterations preceded by 3 warm-up iterations to allow JIT compilation to stabilise. Times are averaged and reported in milliseconds. Four input patterns are tested: *random* (Fisher-Yates shuffle), *ascending*, *descending*, and *partially sorted* (10% random swaps of an otherwise ascending sequence). Input sizes range from $n = 100$ to $n = 10{,}000$.

4.2 Batch Insertion

Tables 1 and 2 show batch insertion times for random and ascending input. Descending and partially sorted inputs show similar trends to ascending, with the Treap consistently matching or outperforming AVL on non-random data.

Table 1: Random input.

n	Treap	AVL	Java TM
100	0.2325	0.1919	0.0993
500	0.2126	0.5174	0.1454
1000	0.3639	0.4904	0.2225
2000	0.7364	0.5235	0.3308
5000	1.3915	1.1918	0.9094
10000	2.4089	2.0659	1.2729

Table 2: Ascending input.

n	Treap	AVL	Java TM
100	0.0058	0.0075	0.0063
500	0.0273	0.0374	0.0265
1000	0.0446	0.0650	0.0554
2000	0.0944	0.1405	0.1630
5000	0.2496	0.3708	0.2851
10000	0.5187	0.7721	0.6523

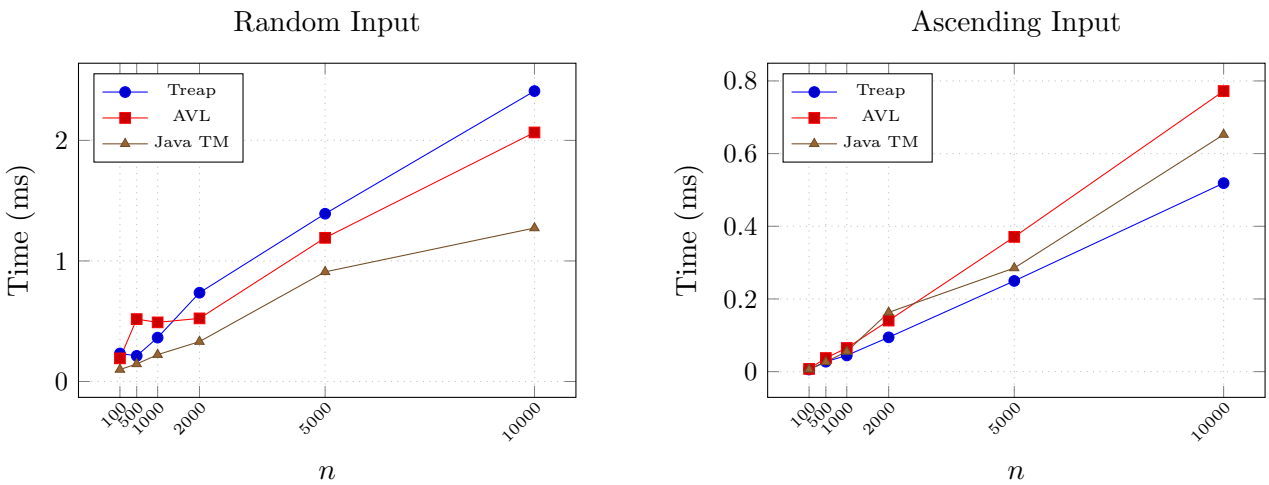


Figure 4: Batch insertion time on random input (left) and ascending input (right).

For random input, all three structures perform similarly. Java's `TreeMap` is the fastest, and the Treap is slightly slower than AVL. This is because the Treap has to generate a random number on every insertion, which adds a small overhead.

For ascending input, the Treap is faster than AVL. At $n = 10{,}000$ the Treap takes 0.52 ms compared to AVL's 0.77 ms. This is because when keys are inserted in order, AVL has to do extra rotations to keep the tree balanced. The Treap does not have this problem because the random priorities keep the tree balanced regardless of the order keys are inserted.

In both cases, all three structures grow at a similar rate as n increases, which confirms that each operation runs in $O(\log n)$ time as expected.

4.3 Search

We tested two types of search. A successful search looks for a key that exists in the tree, while an unsuccessful search looks for a key that does not exist. Tables 3 and 4 show the results for random input.

Table 3: Successful search (ms)

n	Treap	AVL	Java TM
100	0.0872	0.0487	0.0295
500	0.0964	0.0938	0.0862
1000	0.2051	0.2287	0.1389
2000	0.3475	0.3240	0.2789
5000	0.8338	0.5973	0.4601
10000	1.8245	1.4261	0.9967

Table 4: Unsuccessful search (ms)

n	Treap	AVL	Java TM
100	0.0554	0.0395	0.0299
500	0.0606	0.0650	0.1079
1000	0.1310	0.1069	0.1029
2000	0.1902	0.2078	0.1723
5000	0.1712	0.2385	0.1544
10000	0.2435	0.3791	0.1832

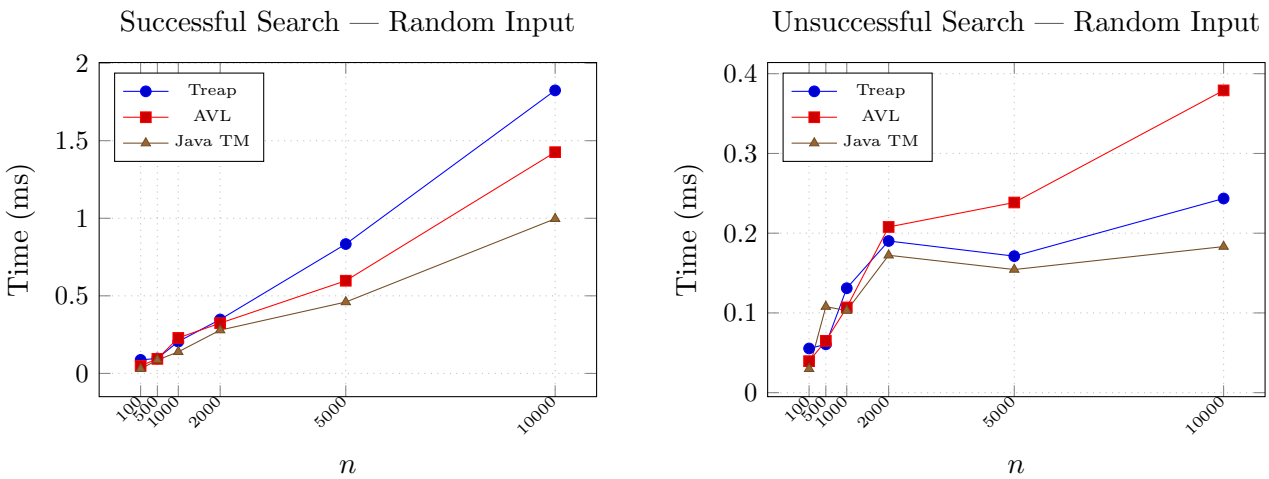


Figure 5: Search time on random input: successful (left) and unsuccessful (right).

For successful search, the Treap is slightly slower than AVL. This is because AVL keeps the tree a bit shorter, so it reaches the target key in fewer steps. Java's `TreeMap` is the fastest in both cases due to its highly optimised implementation.

For unsuccessful search, the Treap is faster than AVL. When a key does not exist, the search has to reach a dead end in the tree. The Treap's random structure spreads these dead ends more evenly, so failed searches do not consistently fall into deeper paths. Interestingly, Java's `TreeMap` is not always the fastest here at $n = 500$, it is slower than both the Treap and AVL, which is likely due to JVM overhead on smaller input sizes.

In both cases all three structures grow at a similar rate as n increases, confirming $O(\log n)$ per operation time.

4.4 Deletion

Tables 5 and 6 show deletion times for random and ascending input.

Table 5: Deletion (ms) — random input.

n	Treap	AVL	Java TM
100	0.1664	0.1530	0.0883
500	0.4283	0.2743	0.1248
1000	0.4935	0.4093	0.2140
2000	1.2847	0.9015	0.3242
5000	0.9116	0.8485	0.6089
10000	2.1797	1.7918	1.2079

Table 6: Deletion (ms) — ascending input.

n	Treap	AVL	Java TM
100	0.0099	0.0080	0.0072
500	0.0304	0.0270	0.0182
1000	0.0557	0.0507	0.0376
2000	0.1170	0.1806	0.1236
5000	0.2778	0.2808	0.2163
10000	0.6212	0.6945	0.4089

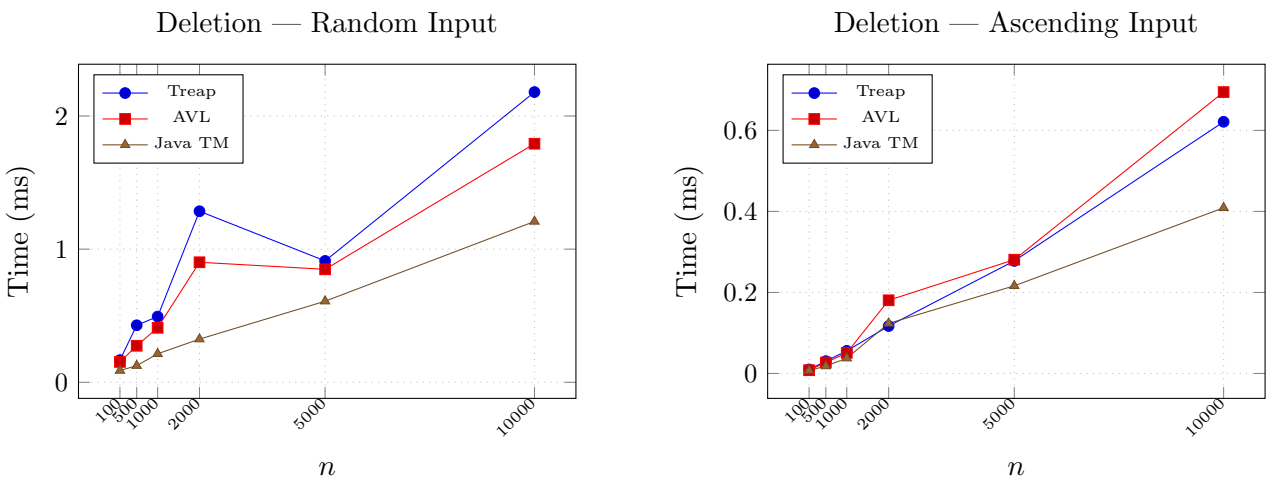


Figure 6: Deletion time on random input (left) and ascending input (right).

For random input, the Treap is slightly slower than AVL for deletion. This is because the Treap has to rotate the node down to a leaf before removing it, which takes extra steps. Java's `TreeMap` is the fastest in both cases.

For ascending input, the Treap is faster than AVL. When keys were inserted in order, AVL had to do a lot of rotations to stay balanced, which makes the tree harder to delete from later. The Treap does not have this problem because the random priorities keep the tree balanced from the start, making deletion easier.

In both cases, all three structures scale at a similar rate as n increases, which confirms that deletion runs in $O(\log n)$ time.

4.5 In-Order Traversal

Tables 7 and 8 show in-order traversal times for random and ascending input.

Table 7: Traversal (ms) — random input.

n	Treap	AVL	Java TM
100	0.0414	0.0590	0.0258
500	0.1808	0.1910	0.0590
1000	0.3581	0.3439	0.1327
2000	0.3459	0.4565	0.1823
5000	0.3620	0.3704	0.1426
10000	0.4207	0.3428	0.1762

Table 8: Traversal (ms) — ascending input.

n	Treap	AVL	Java TM
100	0.0055	0.0071	0.0022
500	0.0172	0.0185	0.0096
1000	0.0374	0.0392	0.0039
2000	0.0663	0.0481	0.0077
5000	0.2191	0.1928	0.0327
10000	0.4083	0.3575	0.1616

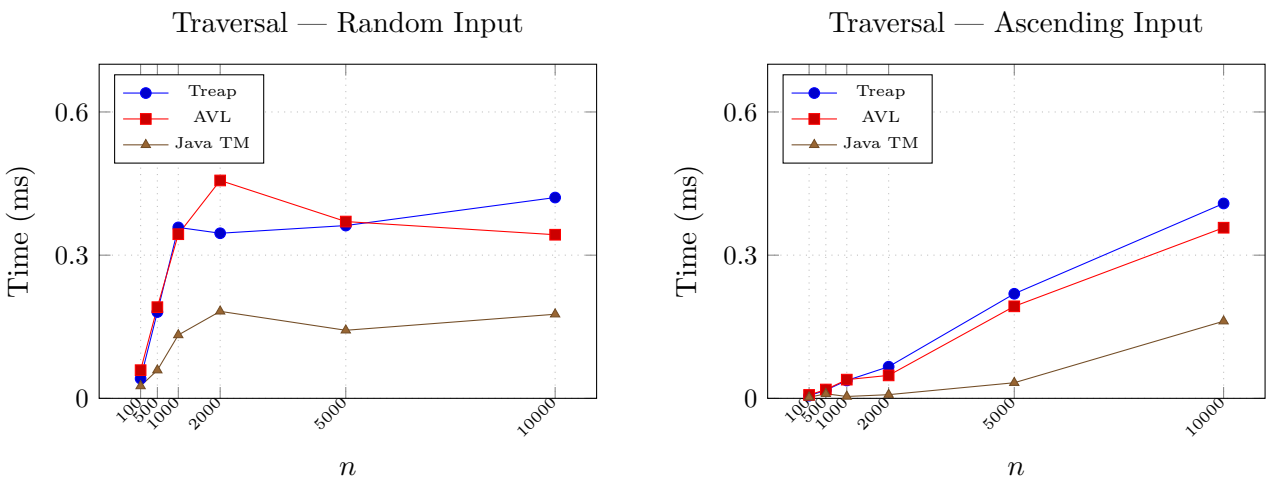


Figure 7: Traversal time on random input (left) and ascending input (right).

In-order traversal visits every node exactly once, so all three structures run in $O(n)$ time. The results confirm this — all three grow at a similar rate as n increases.
The Treap and AVL give very similar times across both input patterns. Java's `TreeMap` is noticeably faster, especially on ascending input, because its iterator is highly optimised and does not have the extra object wrapping used in our course framework.

4.6 Discussion

For all operations and input types, the three data structures show performance of about $O(\log n)$ per operation, which matches what we expect from theory. The main observations are:

- Java's `TreeMap` is the fastest overall, mainly due to JVM optimisations rather than a better algorithm.
- The Treap performs better than the AVL tree on sorted and partially sorted data. This is because its random priorities prevent it from becoming unbalanced, while AVL trees need to perform more rotations when data is inserted in order.
- On random data, the Treap is slightly slower for search operations because it tends to be a bit taller than an AVL tree.
- The Treap is much simpler to implement than an AVL tree. It uses a single rotation process, while AVL trees require multiple rotation cases and extra tracking of node heights.

5 Sorting Algorithms Analysis

5.1 Algorithms

Five sorting algorithms were benchmarked:

(i) **TreapSort** — inserts elements into a Treap and retrieves them in sorted order.

(ii) **PQSort** — uses a priority queue to repeatedly extract the smallest element.

(iii) **TimSort** — a hybrid sorting algorithm used in Java, combining merge and insertion sort.

(iv) **QuickSort** — divides the data around a pivot and sorts recursively.

(v) **MergeSort** — splits the data into halves, sorts them, and merges the results.

5.2 Results

Tables 9 and 10 show the sorting times for random and reverse sorted input.

Table 9: Random input.

n	Treap	PQ	Tim	Quick	Merge
100	0.0213	0.0043	0.0095	0.0017	0.0192
500	0.0958	0.0312	0.0456	0.0068	0.0775
1000	0.2119	0.0829	0.1223	0.0221	0.1926
2000	0.4741	0.2051	0.3340	0.0817	0.4132
5000	1.6464	0.5870	0.6504	0.2424	0.7461
10000	3.3511	1.2686	1.3813	0.5237	1.5597

Table 10: Reverse sorted input.

n	Treap	PQ	Tim	Quick	Merge
100	0.2533	0.0642	0.0069	0.0421	0.0573
500	0.3807	0.1696	0.0302	0.0331	0.1159
1000	0.6383	0.2756	0.0614	0.0705	0.1769
2000	1.1150	0.4874	0.1233	0.0398	1.3153
5000	0.7626	0.4684	0.2904	0.1096	3.2900
10000	1.6309	1.1441	0.2129	0.2345	2.5210

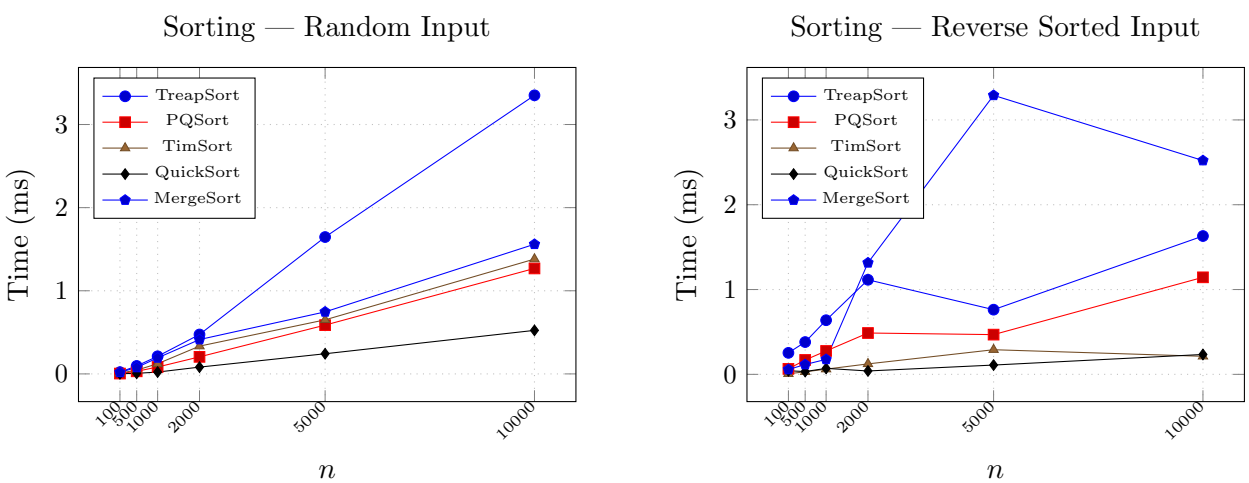


Figure 8: Sorting time on random input (left) and reverse sorted input (right).

5.3 Discussion

TreapSort is the slowest in all cases. Although it runs in $O(n \log n)$, it has extra overhead from random priorities, rotations, and object storage.
QuickSort is the fastest overall, performing well on all input types due to good pivot selection.
TimSort performs very well on ordered data and is similar to MergeSort on random data.
MergeSort is efficient on random input but slow on reverse sorted data due to high memory usage in our implementation.
PQSort has average performance, slower than QuickSort and TimSort but faster than TreapSort.

6 When to Prefer a Treap over an AVLTreeMap

A Treap is often a better choice than an AVL tree in the following cases:

- **Simplicity.** Treaps are much easier to implement. They use a simple rotation loop, while AVL trees require more complex logic and height tracking.
- **Unpredictable input.** Treaps handle all input orders well because their balancing is based on random priorities. AVL trees may need more rotations for sorted data.
- **Fewer rotations.** Treaps usually perform very few rotations per update, which is useful when rotations are expensive.
- **Split and join operations.** Treaps support splitting and joining trees efficiently, which is harder to implement with AVL trees.

An **AVL tree** is better when guaranteed worst-case performance is required, when searches are more important than updates, or when a fully predictable structure is needed.

7 Conclusion

We implemented a generic `TreapMap<K,V>` that successfully passed all tests, including stress tests with 1000 random insertions and deletions, as well as a height test showing that the tree remains balanced even with sorted input.
Our results show that all three tree structures perform in $O(\log n)$ time per operation. The Treap performs particularly well on sorted inputs due to its use of random priorities, which helps keep the tree balanced. While Java's `TreeMap` performs best overall, likely due to built-in optimisations, the Treap remains competitive.
In the sorting benchmark, TreapSort is slower than the other algorithms because of the overhead of maintaining a tree structure. However, it has the advantage of producing a sorted and searchable structure at the same time.